

High-Performance SIMD Software for Spielman Codes in ZKPs

Florian Krieger Christian Dobrouschek Florian Hirner Sujoy Sinha Roy

CHES 2026

- 1 Motivation
- 2 Spielman Codes
- 3 Our Proposed Software Architecture
- 4 Results & Comparison

Motivation

ZKPs & Verifiable Cloud Computing



**Client Device
(Verifier)**

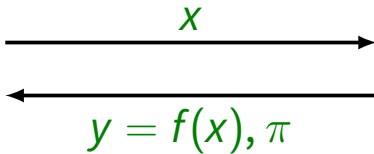


**Cloud Service
(Prover)**

ZKPs & Verifiable Cloud Computing



**Client Device
(Verifier)**

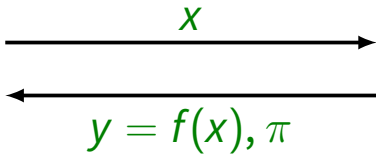


**Cloud Service
(Prover)**

ZKPs & Verifiable Cloud Computing



**Client Device
(Verifier)**



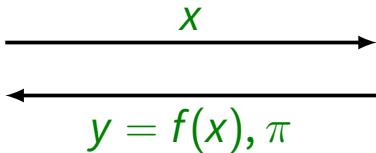
**Cloud Service
(Prover)**

- Increasing number of applications
 - Machine learning
 - Anonymous voting
 - Decentralized finance

ZKPs & Verifiable Cloud Computing



**Client Device
(Verifier)**



Performance & PQ-security



**Cloud Service
(Prover)**

- Increasing number of applications
 - Machine learning
 - Anonymous voting
 - Decentralized finance

The Structure of Zero-Knowledge Proofs

Interactive Oracle Proof

- Non-interactive via FS-transform
- Information-theoretic security

The Structure of Zero-Knowledge Proofs

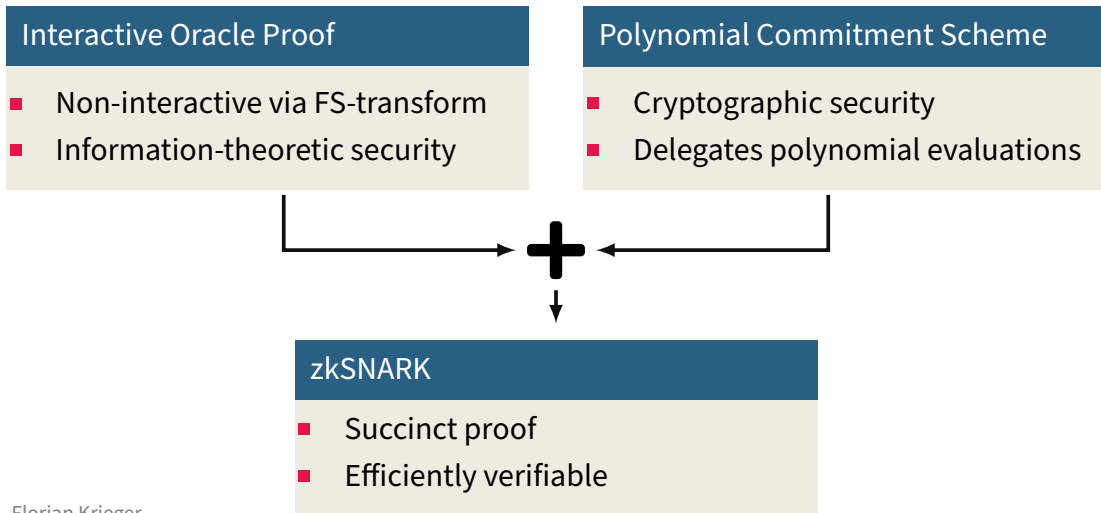
Interactive Oracle Proof

- Non-interactive via FS-transform
- Information-theoretic security

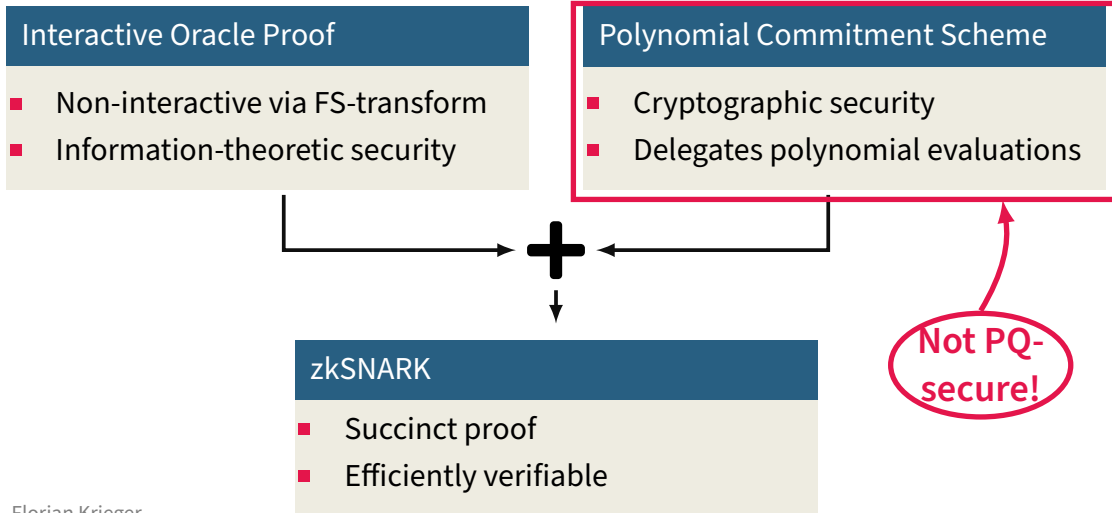
Polynomial Commitment Scheme

- Cryptographic security
- Delegates polynomial evaluations

The Structure of Zero-Knowledge Proofs



The Structure of Zero-Knowledge Proofs



The Brakedown PCS:

- Tensor-IOP based PCS

The Brakedown PCS:

- Tensor-IOP based PCS
- Hash functions for PQ-security

The Brakedown PCS:

- Tensor-IOP based PCS
- Hash functions for PQ-security
- Linear error-correcting codes
 - Operate over gigabytes of data
 - Dominate the latency

The Brakedown PCS:

- Tensor-IOP based PCS
 - Hash functions for PQ-security
 - Linear error-correcting codes
 - Operate over gigabytes of data
 - Dominate the latency
- } Critical for prover performance

The Brakedown PCS: Linear Encoding

$$p(x) = c_0 + c_1x + c_2x^2 + \cdots + c_{N-1}x^{N-1} \quad \leftarrow \text{over field } \mathbb{F} \text{ with } N = k^2$$

The Brakedown PCS: Linear Encoding

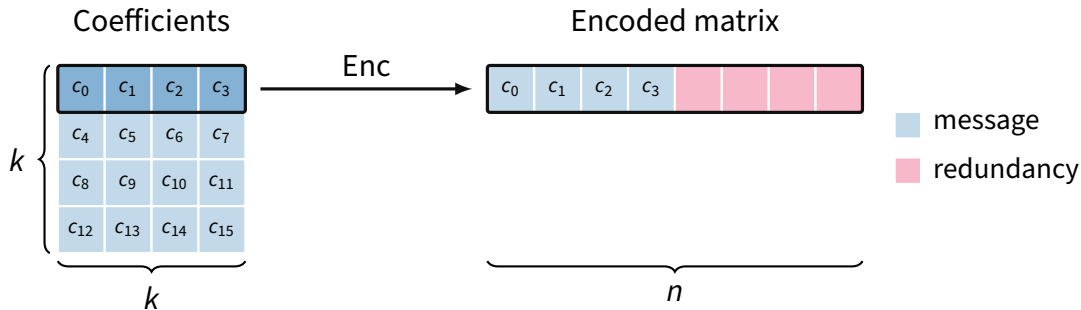
$$p(x) = c_0 + c_1x + c_2x^2 + \cdots + c_{N-1}x^{N-1} \quad \leftarrow \text{over field } \mathbb{F} \text{ with } N = k^2$$

Coefficients

c_0	c_1	c_2	c_3
c_4	c_5	c_6	c_7
c_8	c_9	c_{10}	c_{11}
c_{12}	c_{13}	c_{14}	c_{15}

The Brakedown PCS: Linear Encoding

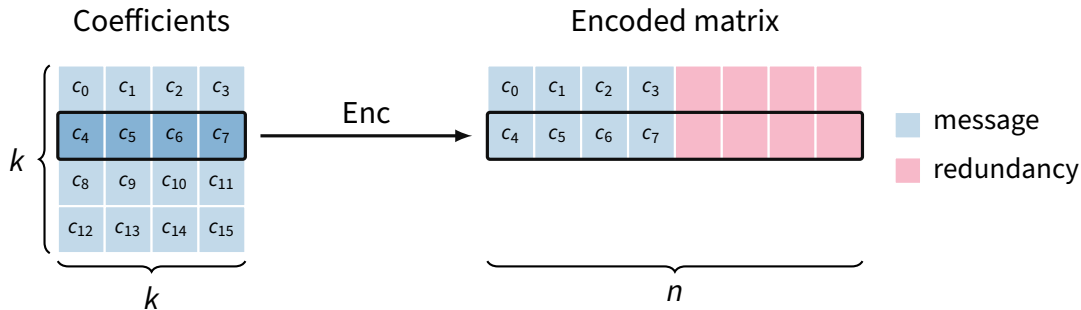
$$p(x) = c_0 + c_1x + c_2x^2 + \dots + c_{N-1}x^{N-1} \quad \leftarrow \text{over field } \mathbb{F} \text{ with } N = k^2$$



$\text{Enc} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ is a linear error-correcting code

The Brakedown PCS: Linear Encoding

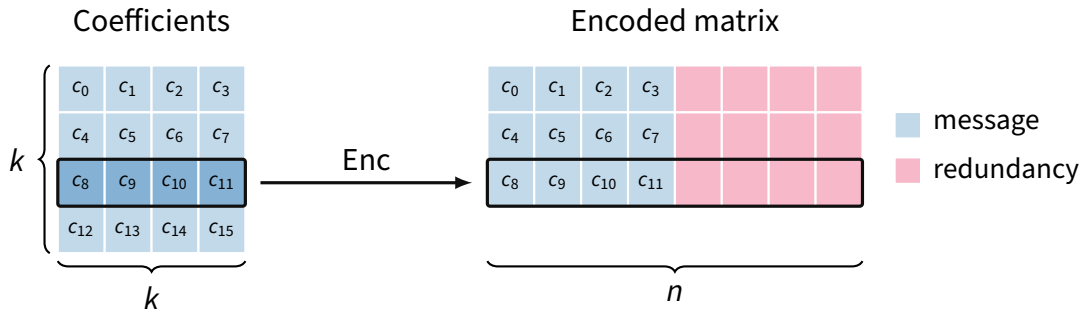
$$p(x) = c_0 + c_1x + c_2x^2 + \dots + c_{N-1}x^{N-1} \quad \leftarrow \text{over field } \mathbb{F} \text{ with } N = k^2$$



$\text{Enc} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ is a linear error-correcting code

The Brakedown PCS: Linear Encoding

$$p(x) = c_0 + c_1x + c_2x^2 + \dots + c_{N-1}x^{N-1} \quad \leftarrow \text{over field } \mathbb{F} \text{ with } N = k^2$$



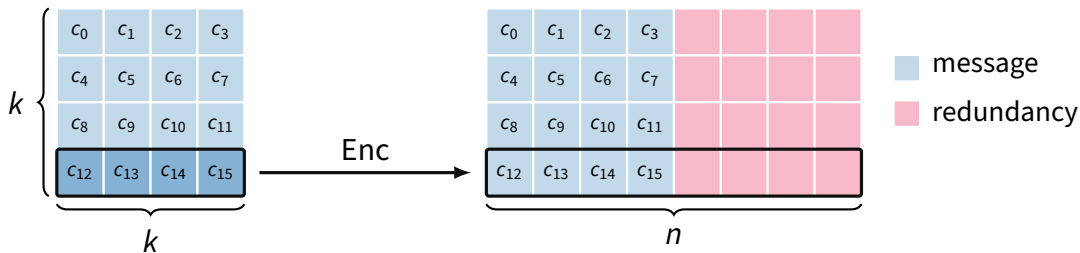
$\text{Enc} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ is a linear error-correcting code

The Brakedown PCS: Linear Encoding

$$p(x) = c_0 + c_1x + c_2x^2 + \dots + c_{N-1}x^{N-1} \quad \leftarrow \text{over field } \mathbb{F} \text{ with } N = k^2$$

Coefficients

Encoded matrix



$\text{Enc} : \mathbb{F}^k \rightarrow \mathbb{F}^n$ is a linear error-correcting code

Reed-Solomon Codes

Spielman Codes

Reed-Solomon Codes

- Rely on NTT
- $\mathcal{O}(N \log N)$
- ❌ Not field agnostic
- ➔ Well explored

Spielman Codes

Reed-Solomon Codes

- Rely on NTT
- $\mathcal{O}(N \log N)$
- ⊖ Not field agnostic
- ➔ Well explored

Spielman Codes

- Less explored
- $\mathcal{O}(N)$
- ⊕ Field agnostic
- ❓ Perceived slower than RS

Reed-Solomon Codes

- Rely on NTT
- $\mathcal{O}(N \log N)$
- ❌ Not field agnostic
- ➔ Well explored

Spielman Codes

- Less explored
- $\mathcal{O}(N)$
- ✅ Field agnostic
- ? ~~Perceived slower than RS~~

We show competitive performance!

Spielman Codes

- Recursive expander graph evaluations

- Recursive expander graph evaluations

input

$$x \in \mathbb{F}^k$$

$$x_0$$

$$x_1$$

$$x_2$$

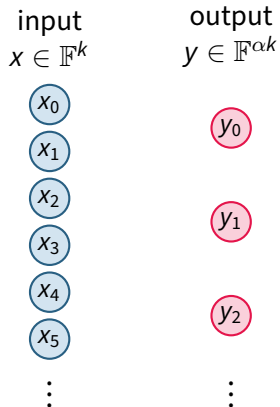
$$x_3$$

$$x_4$$

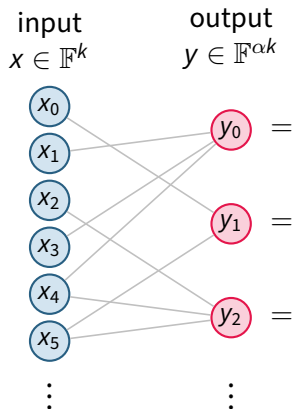
$$x_5$$

⋮

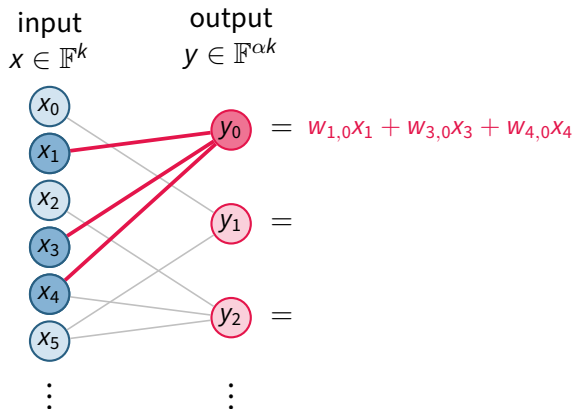
- Recursive expander graph evaluations



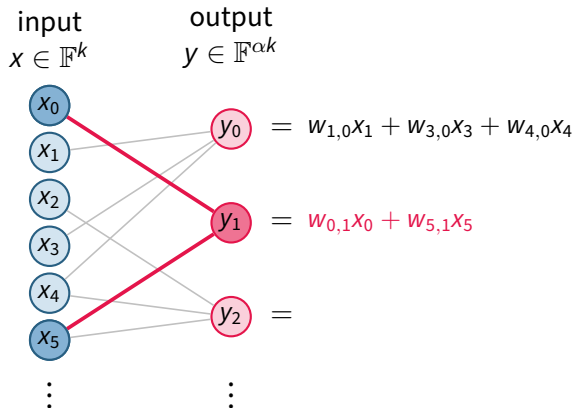
- Recursive expander graph evaluations



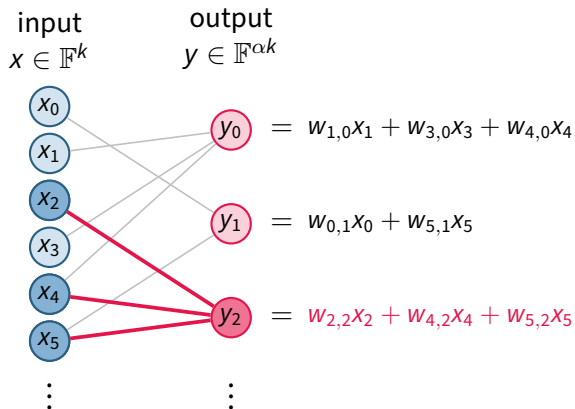
- Recursive expander graph evaluations



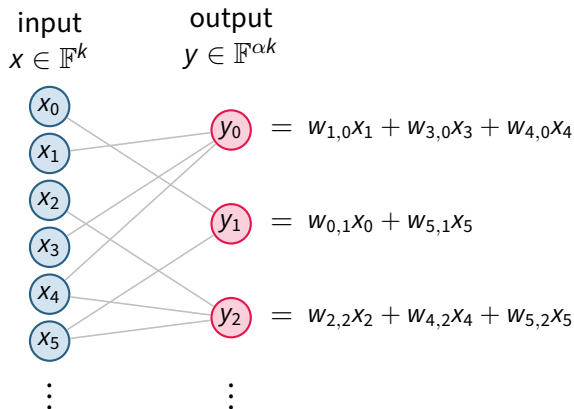
- Recursive expander graph evaluations



- Recursive expander graph evaluations



- Recursive expander graph evaluations
- Random memory access pattern

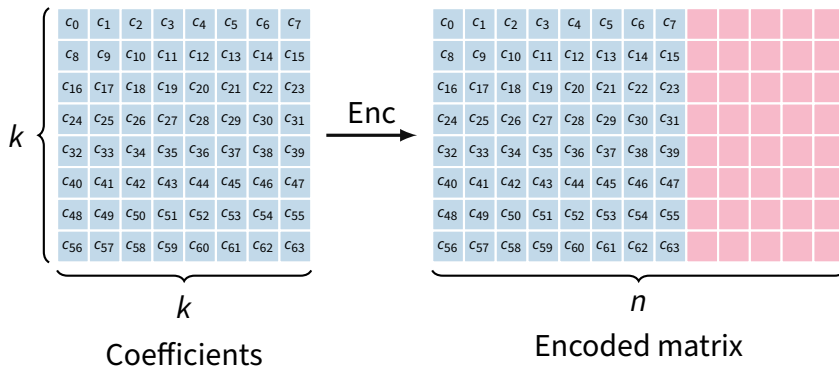


- Non-uniform accesses to large data
 - ➔ How to leverage caches?

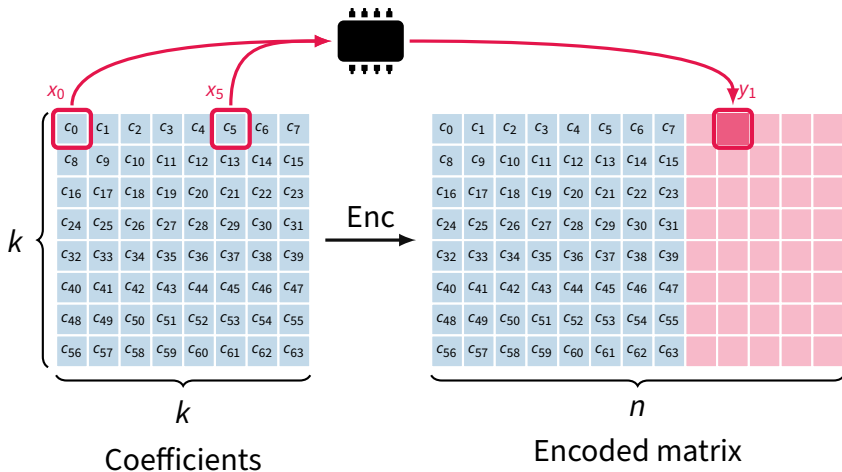
- Non-uniform accesses to large data
 - ➔ How to leverage caches?
- 128-bit field arithmetic
 - ➔ How to accelerate the compute?

- Non-uniform accesses to large data
 - How to leverage caches?
- 128-bit field arithmetic
 - How to accelerate the compute?
- Optimally use CPU resources
 - How to fine-tune for different CPU architectures?

Our Proposed Software Architecture

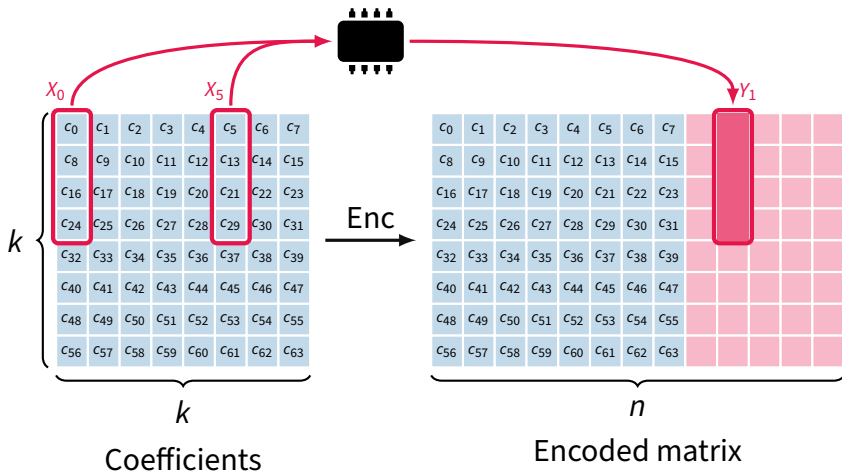


$$y_1 = w_{0,1} \cdot x_0 + w_{5,1} \cdot x_5$$



○ one element (scalar operation)

$$Y_1 = w_{0,1} \cdot X_0 + w_{5,1} \cdot X_5$$

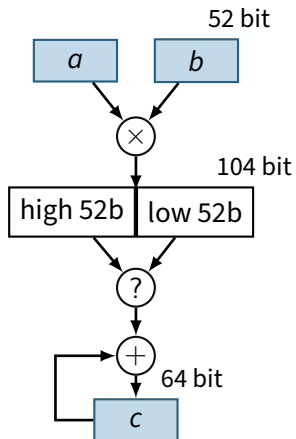


one SIMD register (4 elements)

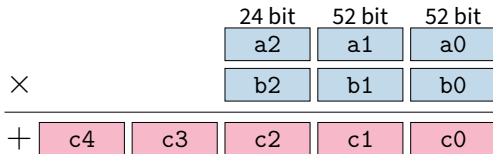
- Consecutive multiply-and-accumulate operations

- Consecutive multiply-and-accumulate operations
- IFMA instructions are a great fit

- Consecutive multiply-and-accumulate operations
- IFMA instructions are a great fit



Optimized Arithmetic



Optimized Arithmetic

		24 bit	52 bit	52 bit	
		a2	a1	a0	
×		b2	b1	b0	
+	c4	c3	c2	c1	c0
+	a1b2 _{hi}	a0b2 _{hi}	a0b1 _{hi}	a0b0 _{hi}	a0b0 _{lo}
+	a2b1 _{hi}	a1b1 _{hi}	a1b0 _{hi}	a0b1 _{lo}	
+	a2b2 _{lo}	a2b0 _{hi}	a0b2 _{lo}	a1b0 _{lo}	
+		a1b2 _{lo}	a1b1 _{lo}		
+		a2b1 _{lo}	a2b0 _{lo}		

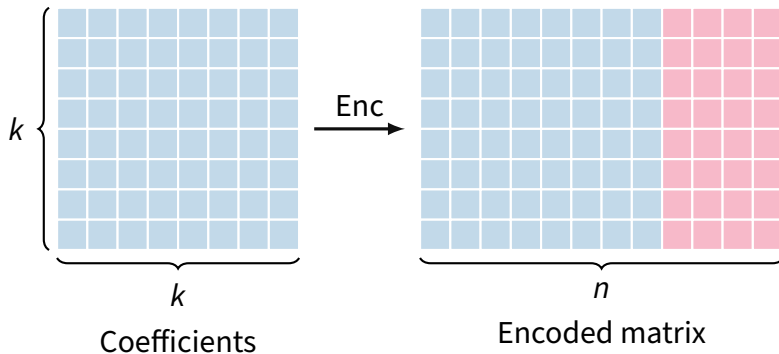
Optimized Arithmetic

			24 bit	52 bit	52 bit
			a2	a1	a0
×			b2	b1	b0
+	c4	c3	c2	c1	c0
+	a1b2 _{hi}	a0b2 _{hi}	a0b1 _{hi}	a0b0 _{hi}	a0b0 _{lo}
+	a2b1 _{hi}	a1b1 _{hi}	a1b0 _{hi}	a0b1 _{lo}	
+	a2b2 _{lo}	a2b0 _{hi}	a0b2 _{lo}	a1b0 _{lo}	
+		a1b2 _{lo}	a1b1 _{lo}		
+		a2b1 _{lo}	a2b0 _{lo}		
=	c4	c3	c2	c1	c0

		24 bit	52 bit	52 bit	
		a2	a1	a0	
×		b2	b1	b0	
+	c4	c3	c2	c1	c0
+	a1b2 _{hi}	a0b2 _{hi}	a0b1 _{hi}	a0b0 _{hi}	a0b0 _{lo}
+	a2b1 _{hi}	a1b1 _{hi}	a1b0 _{hi}	a0b1 _{lo}	
+	a2b2 _{lo}	a2b0 _{hi}	a0b2 _{lo}	a1b0 _{lo}	
+		a1b2 _{lo}	a1b1 _{lo}		
+		a2b1 _{lo}	a2b0 _{lo}		
=	c4	c3	c2	c1	c0

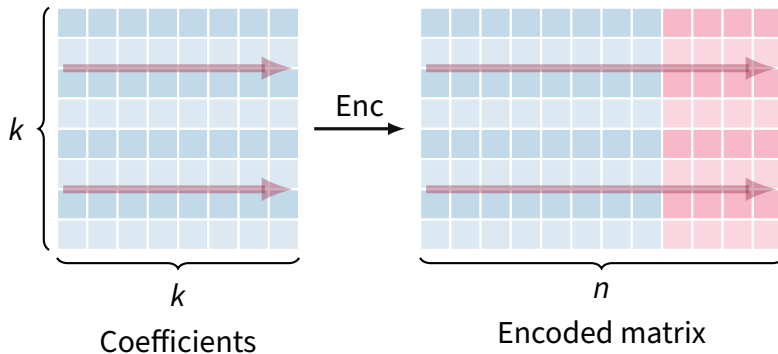
- Modular reduction is expensive!
- ➔ Lazy reduction of Y_i

Cache-Friendly Memory Layout

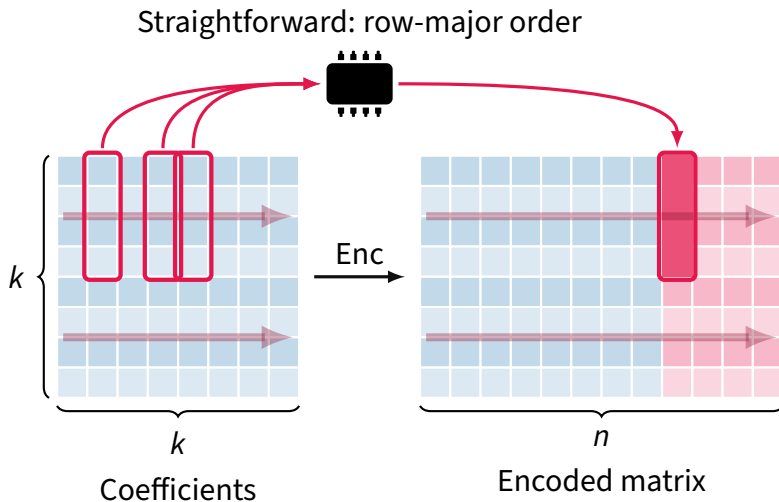


Cache-Friendly Memory Layout

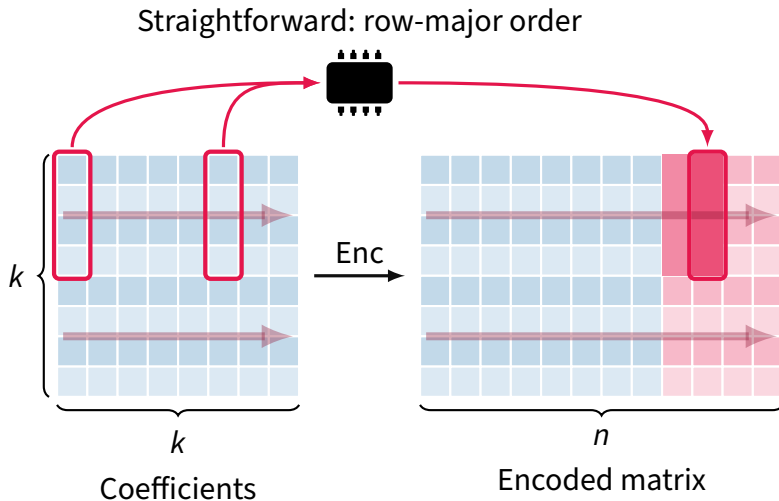
Straightforward: row-major order



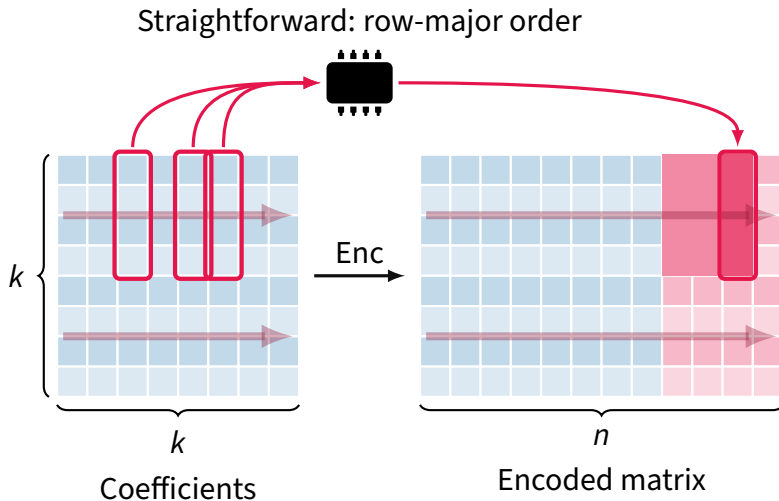
Cache-Friendly Memory Layout



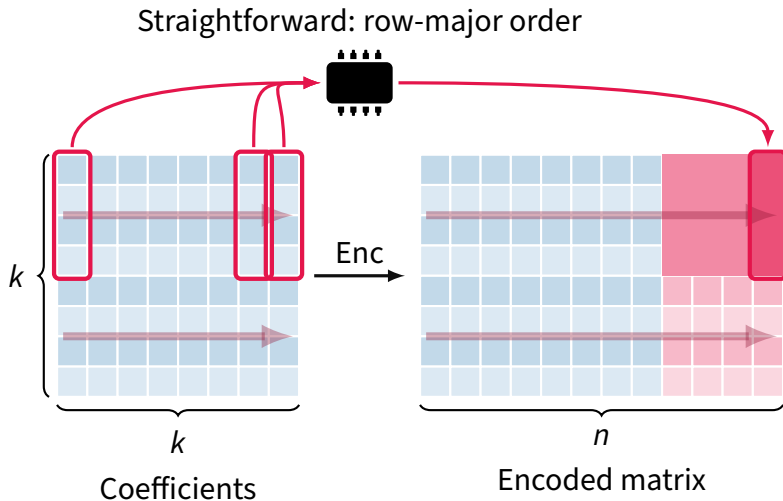
Cache-Friendly Memory Layout



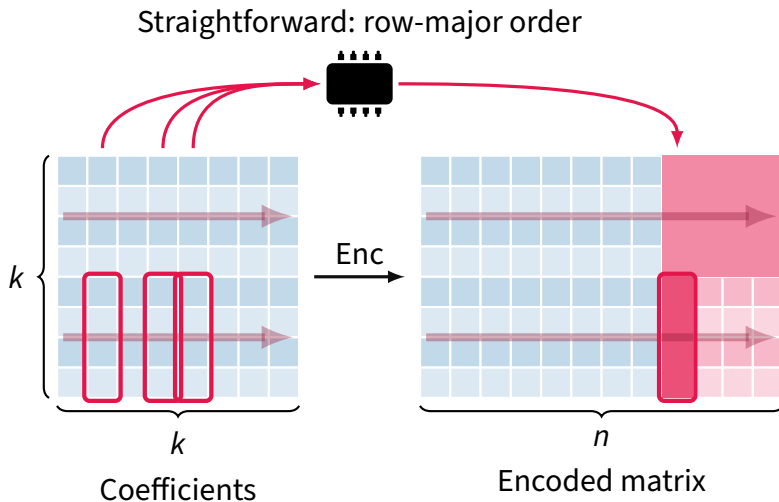
Cache-Friendly Memory Layout



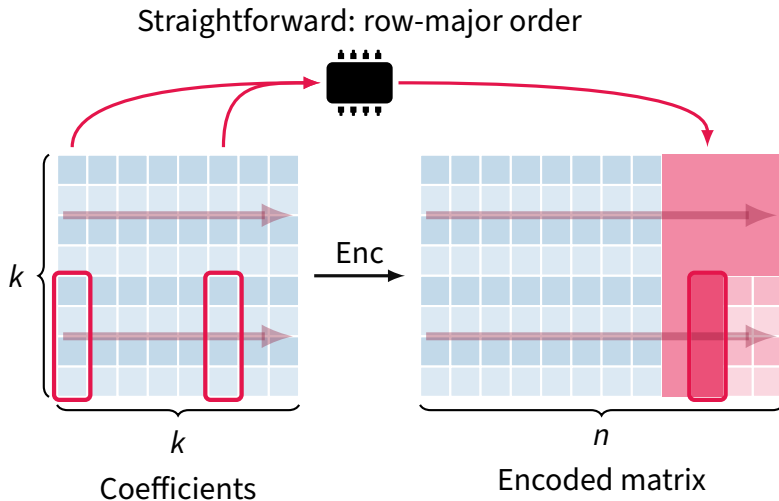
Cache-Friendly Memory Layout



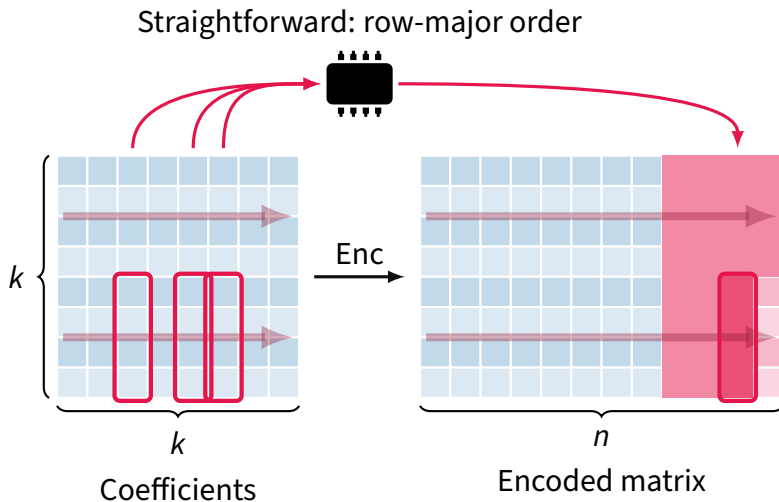
Cache-Friendly Memory Layout



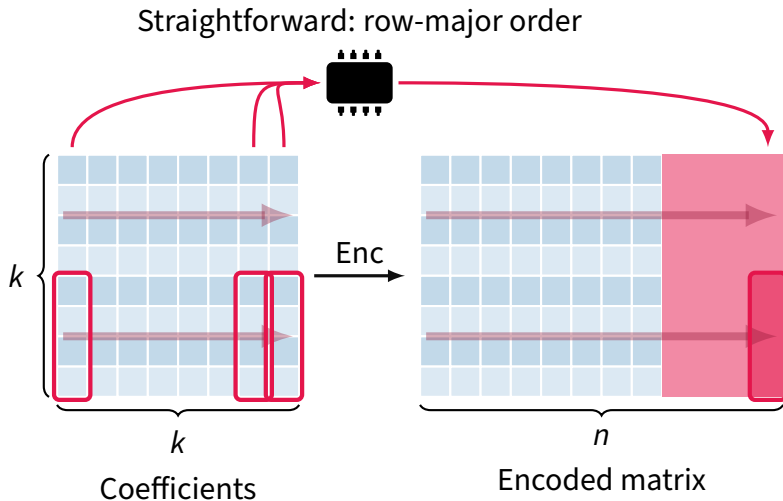
Cache-Friendly Memory Layout



Cache-Friendly Memory Layout

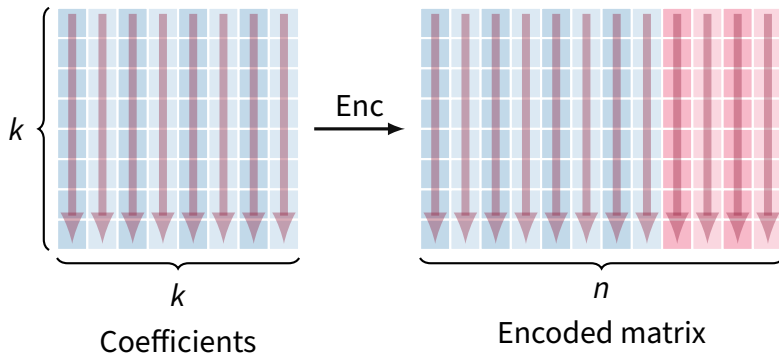


Cache-Friendly Memory Layout

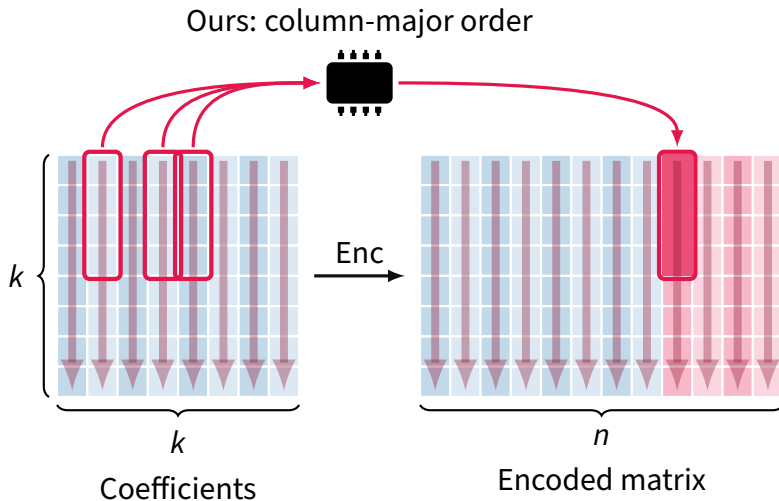


Cache-Friendly Memory Layout

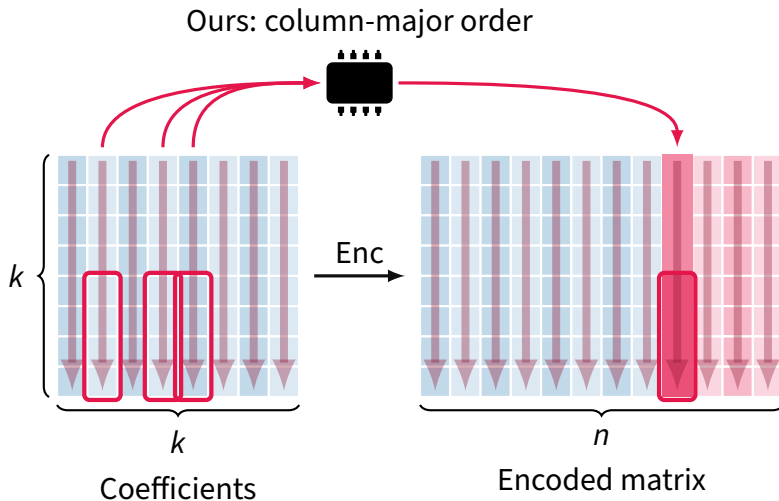
Ours: column-major order



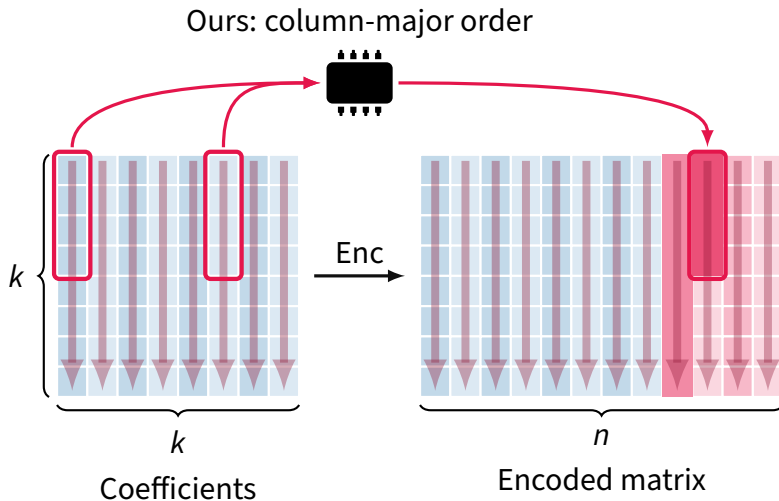
Cache-Friendly Memory Layout



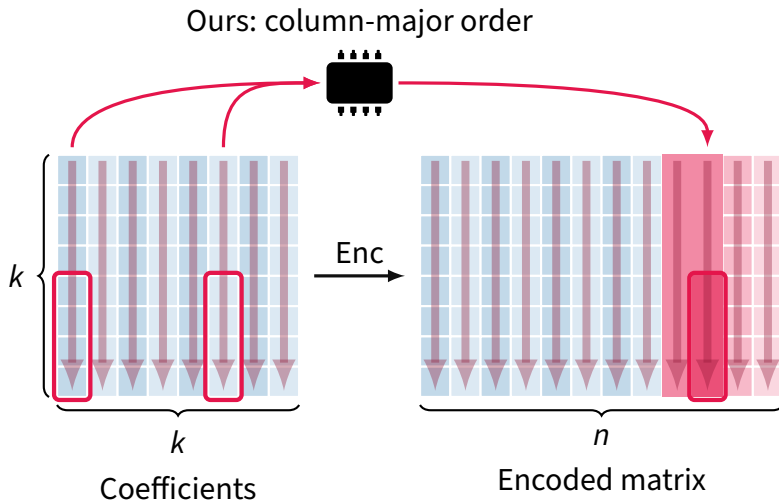
Cache-Friendly Memory Layout



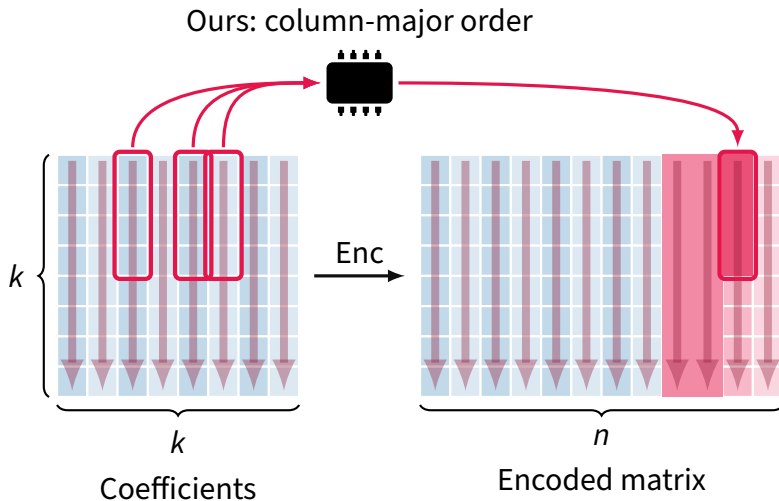
Cache-Friendly Memory Layout



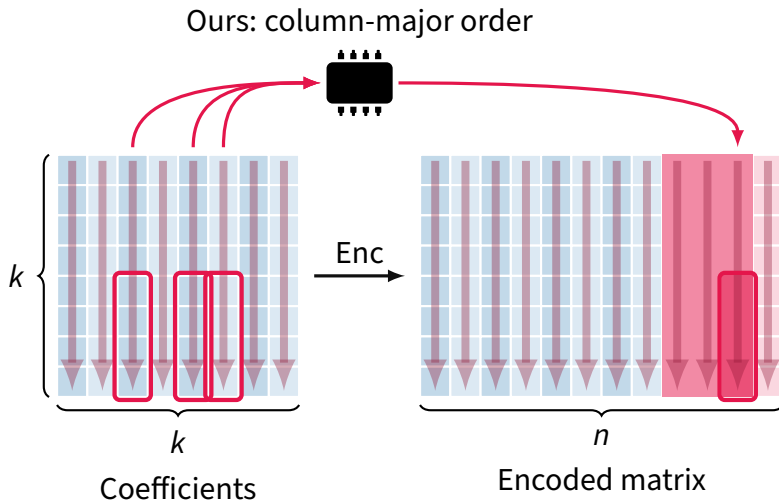
Cache-Friendly Memory Layout



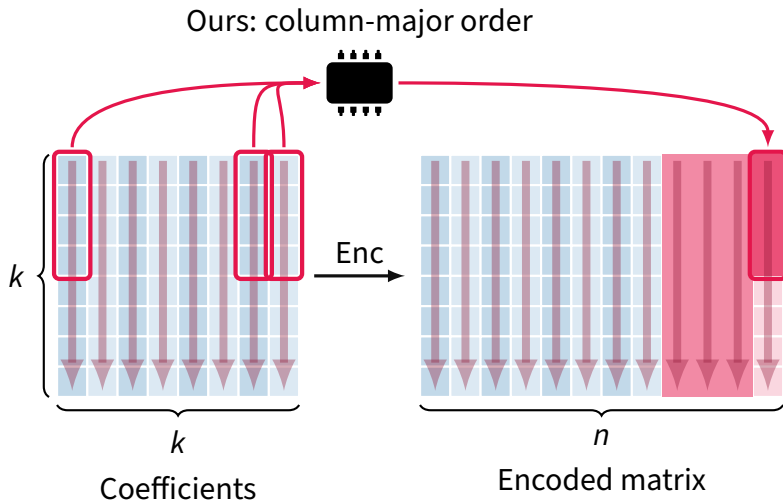
Cache-Friendly Memory Layout



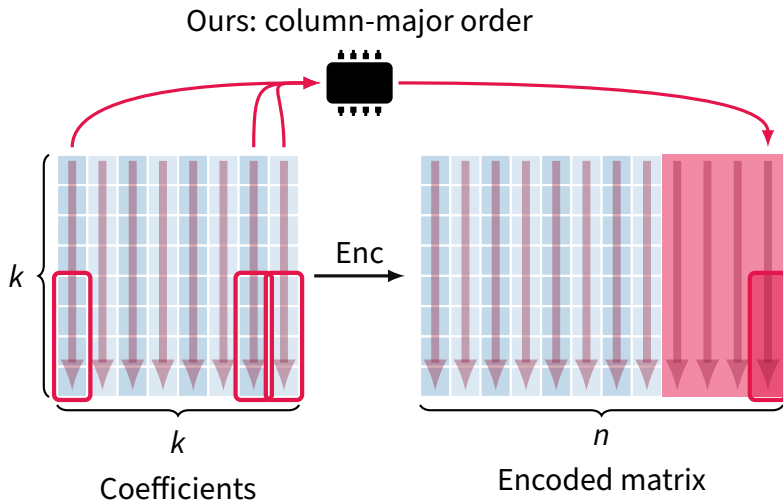
Cache-Friendly Memory Layout



Cache-Friendly Memory Layout



Cache-Friendly Memory Layout



Cache-Friendly Memory Layout

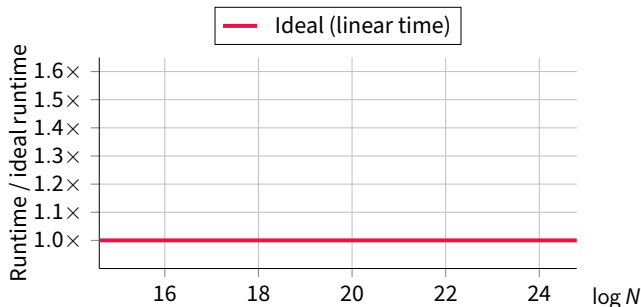
- Spatial cache locality increases performance

Cache-Friendly Memory Layout

- Spatial cache locality increases performance
- But: the performance benefit is not uniform over N

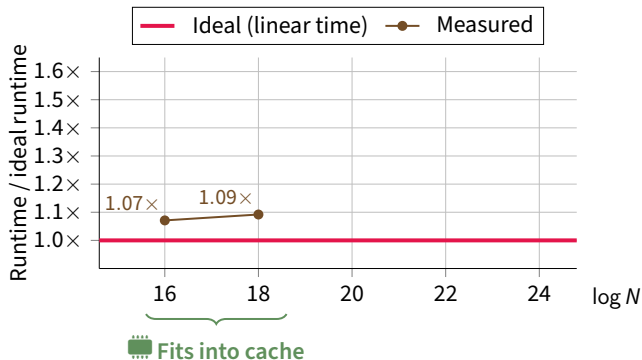
Cache-Friendly Memory Layout

- Spatial cache locality increases performance
- But: the performance benefit is not uniform over N



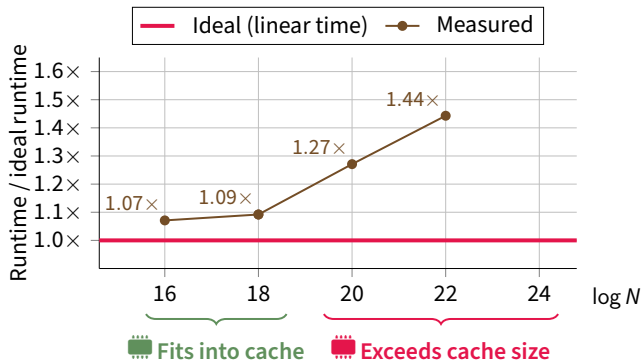
Cache-Friendly Memory Layout

- Spatial cache locality increases performance
- But: the performance benefit is not uniform over N



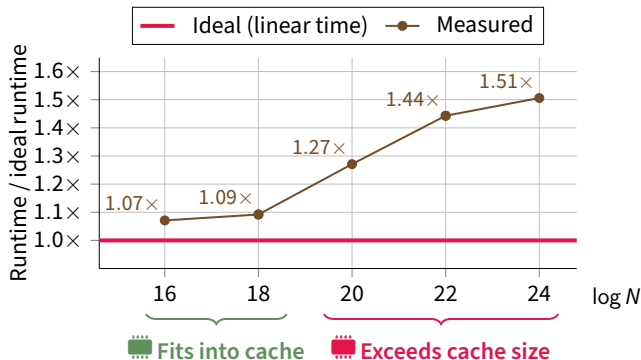
Cache-Friendly Memory Layout

- Spatial cache locality increases performance
- But: the performance benefit is not uniform over N



Cache-Friendly Memory Layout

- Spatial cache locality increases performance
- But: the performance benefit is not uniform over N

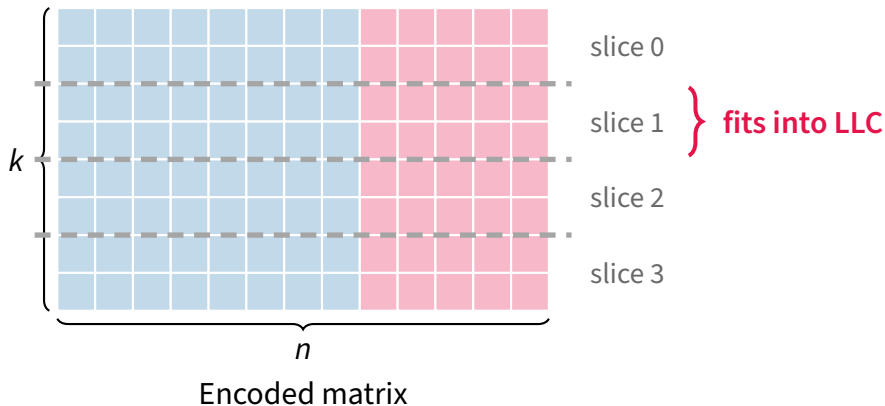


The Slicing Technique

- Improves temporal cache locality

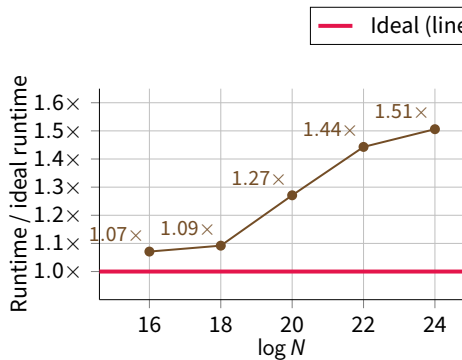
The Slicing Technique

- Improves temporal cache locality
- Horizontal splitting:

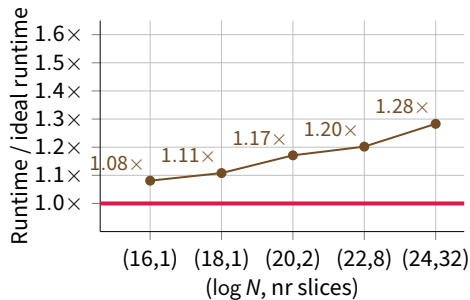


The Slicing Technique

- Each slice fits into cache
- Improves overall performance



Without slicing (Baseline).



With slicing.

- Multithreading

- Multithreading
 - Naive multithreading does not increase the performance

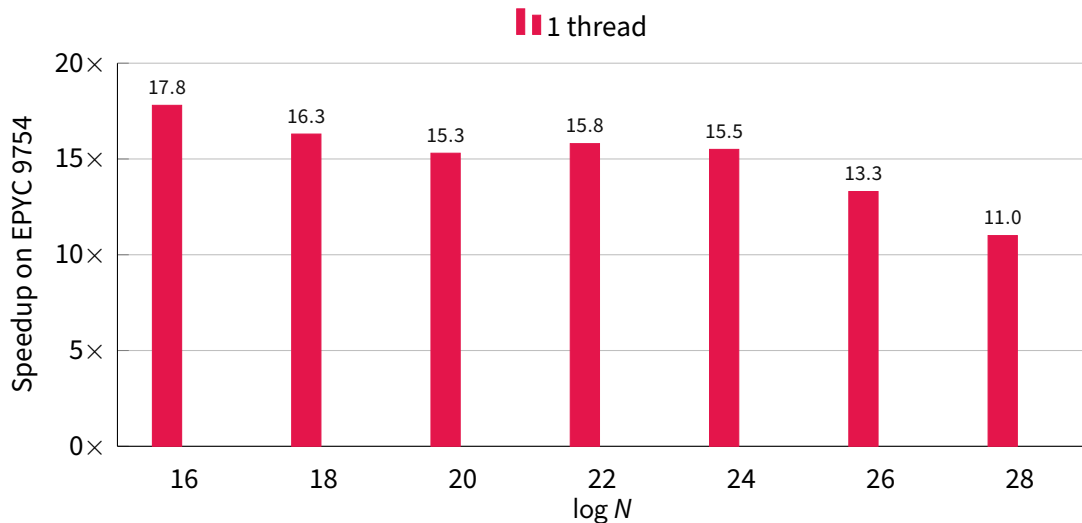
- Multithreading
 - Naive multithreading does not increase the performance
 - Workload distribution methods in the paper

- Multithreading
 - Naive multithreading does not increase the performance
 - Workload distribution methods in the paper
- Fine-tuning parameters is essential for performance
 - Number of slices
 - Number of threads
 - Cache size
 - Cache architecture
 - Guidelines in the paper

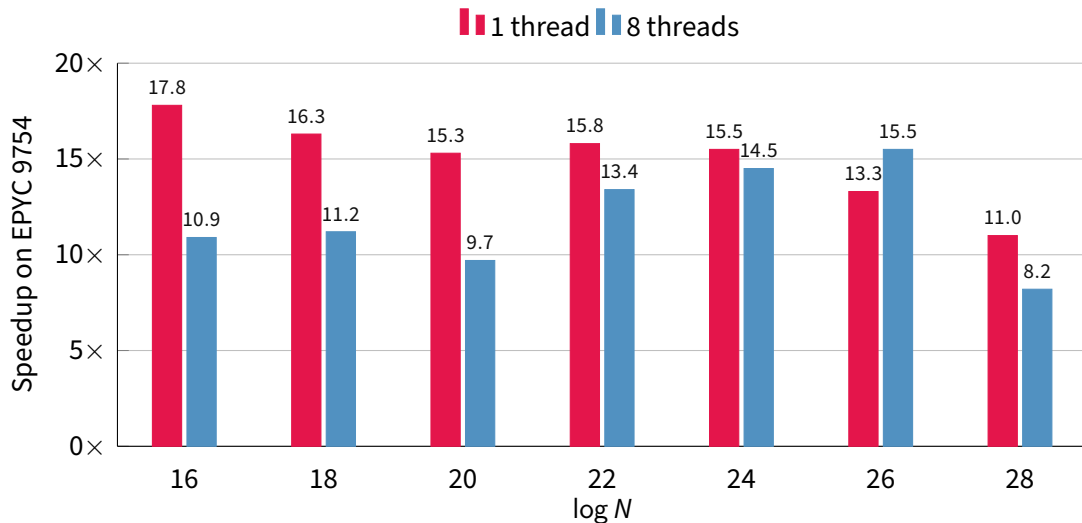
Results & Comparison

- Benchmarked our code on 3 CPUs:
 - Server-grade EPYC 9754
 - Server-grade Xeon Gold 6530
 - Consumer-grade Intel Core i7-11800H
- Similar speedups on all tested CPUs

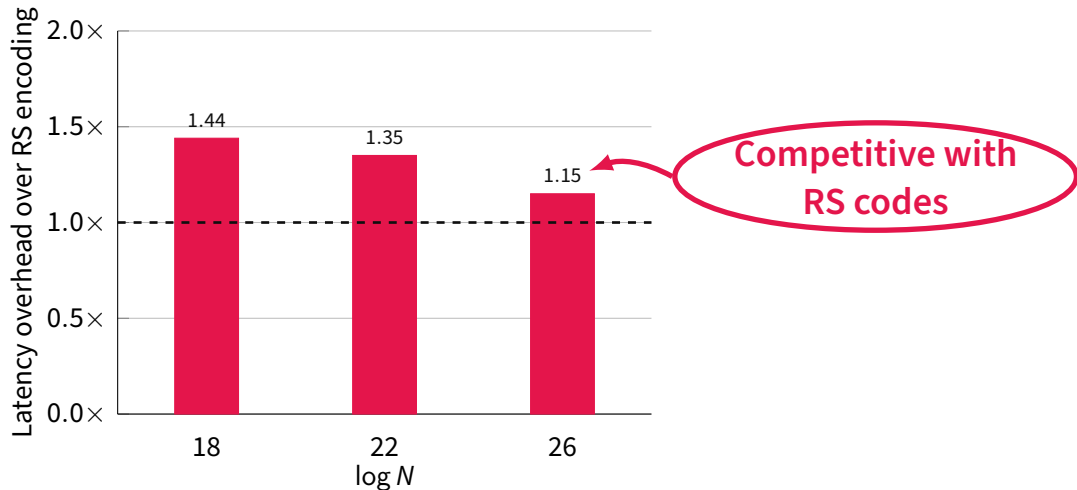
Speedup over Related Software Works



Speedup over Related Software Works



Comparison to Reed-Solomon Codes



High-Performance SIMD Software for Spielman Codes in ZKPs

Florian Krieger Christian Dobrouschek Florian Hirner Sujoy Sinha Roy

CHES 2026